



UNIVERSIDAD DE EL SALVADOR
Facultad Multidisciplinaria De Occidente
Departamento de Ingeniería y Arquitectura
Ingeniería en Desarrollo de Software



Asignatura:

Desarrollo y Técnicas de Aplicaciones Web
Ciclo VII / Cuarto año

Proyecto Final – Registro de Vehículos (CRUD)

Coordinador de cátedra y Tutor GT02:

Ing. Mauricio Giovany Rosales Hernández

Estudiantes:

Ricardo Adan Patiño Hernández

Alexandra Quinteros Cárcamo

Gabriel Ernesto Díaz Galdámez

Víctor Noé Rodas Rivera

Carnet:

PH23011

QC23006

DG22023

RR23027

Fecha:

13 de Junio de 2026

Índice de Contenidos

Introducción.....	1
Objetivos.....	1
Objetivo General.....	1
Objetivos Específicos.....	1
Descripción del Sistema Web.....	1
Problema que Resuelve.....	1
Arquitectura General.....	2
Orquestación del Ciclo de Vida.....	2
Tecnologías Utilizadas.....	2
Implementación de Requisitos Obligatorios	3
Manipulación del DOM y Validación — js/ui/.....	3
js/ui/form-handler.js.....	3
js/ui/navigation.js.....	3
Operaciones CRUD y Persistencia — js/crud/.....	3
js/crud/storage.js.....	3
js/crud/crud.js.....	4
Validación de Formulario — js/utls/validations.js.....	4
Procesamiento en Segundo Plano — js/dashboard/.....	4
js/dashboard/metrics-worker.js.....	4
js/dashboard/dashboard.js	5
Consumo de API Externa — js/api/.....	5
js/api/geolocation.js.....	5
js/api/weather.js.....	6
Gestión de Sesión — js/utls/session.js.....	6
Conclusiones	6

Introducción

El presente documento constituye el reporte técnico del proyecto final desarrollado para la asignatura DTW135 — Desarrollo de Tecnologías Web. El proyecto consiste en una aplicación web de **Registro de Vehículos** construida íntegramente sobre tecnologías web nativas: JavaScript Vanilla con módulos ES6+, HTML5 semántico y hojas de estilo CSS con Custom Properties.

El sistema permite a los usuarios gestionar un inventario vehicular completo a través de operaciones CRUD, visualizar métricas calculadas en segundo plano mediante un Web Worker dedicado, consultar condiciones climáticas en tiempo real integrando la API pública Open-Meteo, y detectar la ubicación geográfica del usuario a través de la Geolocation API del navegador.

El desarrollo se organizó a través de múltiples ramas de trabajo en Git: feature/logica-crud, feature/interfaz-ui, feat/ui-app y feature/testing. Esta última actuó como entorno de integración y refactorización, donde se formalizaron patrones clave —como el módulo de validaciones puras y el orquestador central— antes de fusionarse con main.

Objetivos

Objetivo General

Desarrollar una aplicación web funcional de gestión de inventario vehicular que aplique de forma integrada los conceptos fundamentales de Desarrollo de Tecnologías Web: manipulación avanzada del DOM, consumo de APIs externas, procesamiento en segundo plano con Web Workers y gestión de estado mediante Web Storage.

Objetivos Específicos

1. Implementar una **arquitectura modular** con ES Modules que separe responsabilidades en capas claramente definidas: presentación (js/ui/), lógica de negocio (js/crud/), integración con servicios externos (js/api/) y orquestación del ciclo de vida (js/app/).
2. Diseñar y construir un **CRUD completo** para vehículos con persistencia en localStorage, incluyendo validación de datos del lado del cliente con mensajes descriptivos al usuario.
3. Integrar la **Web Workers API** para desacoplar el cálculo de métricas del hilo principal de la interfaz, evitando bloqueos de renderizado durante operaciones de cómputo.
4. Consumir la **API REST pública Open-Meteo** de forma asíncrona con fetch() y async/await para mostrar condiciones climáticas en tiempo real.
5. Implementar la **Geolocation API** del navegador para obtener coordenadas del usuario y utilizarlas como parámetros dinámicos en la consulta climática, con un fallback geográfico ante permisos denegados.
6. Gestionar la **sesión activa** del usuario mediante sessionStorage, registrando el tiempo transcurrido desde el inicio de la visita con actualización en tiempo real.

Descripción del Sistema Web

Problema que Resuelve

Las instituciones que administran flotas o colecciones de vehículos necesitan una herramienta ágil para registrar, consultar, editar y eliminar información vehicular sin depender de infraestructura de servidor costosa. La aplicación proporciona esta funcionalidad de forma completamente cliente-side, operando en cualquier navegador moderno sin necesidad de backend, base de datos remota ni instalación de dependencias adicionales.

Arquitectura General

La aplicación sigue un modelo de **Single Page Application (SPA) sin framework**. El único archivo HTML (index.html) actúa como contenedor de tres vistas: #dashboard, #registration e #inventory. Solo una vista es visible a la vez, controlada mediante manipulación directa del DOM sin recargar la página. La estructura de módulos es la siguiente:

```
DtwRegistroVehiculosCRUD/
├── index.html
├── css/
│   └── styles.css
├── js/
│   ├── app/
│   │   └── main.js          (Orquestador central)
│   ├── ui/
│   │   ├── navigation.js   (Control de navegación)
│   │   ├── form-handler.js  (Gestión del formulario)
│   │   └── table-render.js  (Renderizado del inventario)
│   ├── crud/
│   │   ├── crud.js         (Operaciones CRUD)
│   │   └── storage.js       (Abstracción de localStorage)
│   ├── api/
│   │   ├── geolocation.js   (Geolocation API + fallback)
│   │   └── weather.js       (Fetch a Open-Meteo)
│   ├── dashboard/
│   │   ├── dashboard.js     (Coordinador del Worker)
│   │   └── metrics-worker.js (Web Worker de métricas)
│   └── utils/
│       ├── session.js       (Gestión de sessionStorage)
│       └── validations.js    (Validaciones puras)
```

Orquestación del Ciclo de Vida

El módulo js/app/main.js centraliza la inicialización. Al dispararse el evento DOMContentLoaded, ejecuta en secuencia:

```
DOMContentLoaded
├── initNavigation()    → Listeners del sidebar; controla visibilidad de secciones
├── initMetricsWorker() → Instancia el Worker; configura onmessage para el DOM
├── initWeatherAndGeo() → Solicita GPS; llama a fetchWeather(); renderiza temperatura
├── initSession()       → Inicia temporizador de sesión con sessionStorage
├── refreshList()        → Lee localStorage; genera filas del inventario
├── initForm()          → Vincula submit; valida; persiste; refresca lista y dashboard
└── updateDashboard()   → Envía datos al Worker; responde con métricas calculadas
```

Todo el bloque está envuelto en un try/catch global para capturar errores críticos de inicialización y notificar al usuario.

Tecnologías Utilizadas

#	Tecnología / API	Especificación	Rol en el Sistema
1	JavaScript Vanilla	ES2020+ (ESModules)	Lógica completa de la aplicación
2	Web Workers API	WHATWG Living Standard	Cálculo de métricas en hilo secundario
3	Fetch API	WHATWG Living Standard	Consumo de la API climática Open-Meteo
4	Geolocation API	W3C Geolocation API	Detección de coordenadas GPS del usuario
5	localStorage	Web Storage API	Persistencia permanente del inventario
6	sessionStorage	Web Storage API	Registro del timestamp de inicio de sesión
7	HTML5 Semántico	HTML Living Standard	Estructura SPA (aside, header, section)

8	CSS Custom Properties	CSS Variables nivel 4	Sistema de tokens de diseño reutilizables
9	Font Awesome 6	CDN v6.0.0	Iconografía visual de la interfaz
10	Open-Meteo API	REST / JSON (gratuita, sin clave)	Datos meteorológicos en tiempo real
11	Git / GitHub	VCS	Control de versiones y colaboración por ramas

Implementación de Requisitos Obligatorios

Manipulación del DOM y Validación — js/ui/

js/ui/form-handler.js

Gestiona el ciclo completo del formulario de registro. Utiliza `new FormData(form)` para extraer los campos y construye el objeto vehículo con las propiedades `brand`, `model`, `year` y `fuel`. La lógica de validación se delega completamente al módulo `js/utls/validations.js`, manteniendo el formulario limpio y sin reglas de negocio embebidas.

Implementa el patrón de **edición en el mismo formulario**: al hacer clic en “Editar”, la función `handleEdit(id)` recupera el vehículo por ID desde `localStorage`, puebla los campos mediante asignación directa (`form.brand.value = vehicle.brand`) y modifica el texto del botón submit. Al guardar, invoca los callbacks `refreshList()` y `updateDashboard()` inyectados como parámetros, garantizando que la UI se mantenga sincronizada.

// Patrón: inyección de dependencias mediante callbacks

```
export function initForm(refreshList, refreshDashboard) {
  const form = document.getElementById('vehicle-form');
  form.addEventListener('submit', event => {
    event.preventDefault();
    const vehicle = { brand, model, year, fuel }; // extraído con FormData
    validateVehicle(vehicle); // lanza Error si inválido
    processVehicle(vehicle, editingId);
    refreshList();
    refreshDashboard();
  });
}
```

js/ui/table-render.js

Genera el inventario dinámicamente con `document.createElement('tr')` e inyecta el HTML de cada fila mediante `template literals`. Los botones de acción (Editar / Eliminar) reciben sus callbacks como parámetros de la función `renderVehicleList(editCallback, deleteCallback)`, aplicando el mismo patrón de inyección de dependencias para mantener el renderizador completamente desacoplado de la lógica de negocio.

js/ui/navigation.js

Implementa la navegación SPA iterando sobre los elementos `.sidebar nav a` y registrando listeners que ocultan todas las secciones `.view-section` para luego mostrar únicamente la sección destino, cuyo ID se extrae del atributo `href` con `.getAttribute('href').substring(1)`.

Operaciones CRUD y Persistencia — js/crud/

js/crud/storage.js

Encapsula el acceso a `localStorage` bajo la clave constante `'dtw_vehicles'`. Expone dos funciones puras:

- `getVehicles()` — Deserializa el JSON almacenado; retorna arreglo vacío si no existe dato.

- `saveVehicles(vehicles)` — Serializa y persiste; lanza un error descriptivo ante `QuotaExceededError`.

La extracción de `STORAGE_KEY` como constante de módulo fue un hallazgo directo de las pruebas realizadas en la rama `feature/testing`, donde se detectó que referencias literales dispersas dificultaban el mantenimiento.

js/crud/crud.js

Implementa las tres operaciones de escritura sobre el arreglo deserializado:

- **Crear:** genera un ID único con `Date.now().toString()` y hace push al arreglo.
- **Actualizar:** localiza el índice con `findIndex()` y reemplaza el objeto usando el operador spread `{...vehicleData, id}`.
- **Eliminar:** filtra el arreglo con `.filter(v => v.id !== id)`.

Cada operación finaliza llamando a `saveVehicles()` para persistir el estado.

Validación de Formulario — js/utils/validations.js

La función `validateVehicle(vehicle)` aplica validaciones secuenciales con lanzamiento de excepciones tipadas:

```
export function validateVehicle(vehicle) {

  if (!vehicle.brand || vehicle.brand.trim() === '') {
    throw new Error('La marca es obligatoria');
  }

  if (!vehicle.model || vehicle.model.trim() === '') {
    throw new Error('El modelo es obligatorio');
  }

  if (!vehicle.year) {
    throw new Error('El año es obligatorio');
  }

  const year = parseInt(vehicle.year);

  if (year < 1900 || year > 2027) {
    throw new Error('Año inválido');
  }

  if (!vehicle.fuel) {
    throw new Error('Debe seleccionar combustible');
  }

  return true;
}
```

Las funciones son **puras** —sin efectos secundarios ni acceso al DOM— lo que las hace directamente reutilizables y testeables de forma aislada. Este diseño emergió del refactor documentado en la rama `feature/testing` (commit: "refactor: refactorizar funciones de validacion para hacerlas puras y reutilizables").

Procesamiento en Segundo Plano — js/dashboard/

js/dashboard/metrics-worker.js

Es un Web Worker independiente que opera en un hilo del navegador separado del hilo principal de la UI. Recibe el arreglo completo de vehículos mediante el evento `message`, valida que sea un Array y calcula el siguiente objeto de métricas:

```
self.onmessage = function (e) {
```

```

const vehicles = e.data;

if (!Array.isArray(vehicles)) {
  self.postMessage({
    error: 'Data is not an array'
  });
  return;
}

const metrics = {
  total: vehicles.length,

  gasolina: vehicles.filter(
    vehicle => vehicle.fuel === 'Gasolina'
  ).length,

  electrico: vehicles.filter(
    vehicle => vehicle.fuel === 'Electrico'
  ).length,

  diesel: vehicles.filter(
    vehicle => vehicle.fuel === 'Diesel'
  ).length,

  hibrido: vehicles.filter(
    vehicle => vehicle.fuel === 'Hibrido'
  ).length,

  avgYear: vehicles.length > 0
    ? Math.round(
        vehicles.reduce(
          (acc, vehicle) =>
            acc + parseInt(vehicle.year),
          0
        ) / vehicles.length
      )
    : 0
};

self.postMessage(metrics);
};

```

El Worker calcula **seis métricas** (cinco tipos de combustible más el año promedio de la flota), aunque el dashboard actual expone tres. Esto evidencia que el diseño está preparado para ampliar las visualizaciones sin modificar el Worker.

js/dashboard/dashboard.js

Coordina la comunicación con el Worker. `initMetricsWorker()` verifica `window.Worker` antes de instanciar, y define `metricsWorker.onmessage` para actualizar los elementos del DOM al recibir la respuesta. `updateDashboard()` incluye un **fallback síncrono** que calcula las métricas directamente en el hilo principal para navegadores sin soporte de Workers, garantizando que la funcionalidad esté disponible en cualquier entorno.

Consumo de API Externa — js/api/

js/api/geolocation.js

Consumes `navigator.geolocation.getCurrentPosition()`. Si el usuario concede el permiso, muestra las coordenadas en el footer y llama a `fetchWeather(lat, lon)` con los datos reales. Si deniega el acceso o el

navegador no soporta la API, utiliza un **fallback geográfico predefinido** (coordenadas de Madrid: 40.4168, -3.7038) para garantizar que el widget climático siempre muestre datos.

js/api/weather.js

Realiza la petición a la API pública Open-Meteo mediante `async/await`:

```
const response = await fetch(
  `https://api.open-meteo.com/v1/forecast?latitude=${lat}` +
  `&longitude=${lon}&current=temperature_2m`
);
```

Verifica `response.ok` antes de parsear el JSON, extrae `data.current.temperature_2m`, y renderiza la temperatura redondeada con `Math.round()` en el widget del header. Ante cualquier fallo de red, actualiza el widget con un ícono de advertencia, manteniendo la UI coherente.

Gestión de Sesión — js/utils/session.js

`initSession()` registra `Date.now()` en `sessionStorage` bajo la clave `'session_start'` únicamente si la clave no existe previamente, lo que preserva el tiempo correcto incluso si el usuario recarga la página dentro de la misma sesión de navegador. Un `setInterval` de 1000 ms calcula el tiempo transcurrido, lo formatea como MM:SS usando `padStart(2, '0')` y actualiza el elemento `#session-timer` del header.

Conclusiones

La arquitectura modular adoptada en este proyecto demostró ser una decisión de diseño acertada para una aplicación JavaScript Vanilla de esta complejidad. La separación en capas (`ui/`, `crud/`, `api/`, `dashboard/`, `utils/`) permitió que cada integrante del equipo trabajara en su módulo de forma paralela con mínimos conflictos de integración, tal como lo evidencia el historial de ramas en Git.

La implementación del **Web Worker** para el cálculo de métricas hizo explícita la separación entre el hilo de la interfaz y el hilo de cómputo, y evidenció el modelo de comunicación asíncrona por mensajes (`postMessage` / `onmessage`) sin memoria compartida. El fallback síncrono implementado garantiza que la aplicación sea funcional en cualquier entorno.

El **patrón de orquestación central** en `js/app/main.js` simplificó enormemente la depuración: al existir un único punto de entrada que inicializa todos los módulos en orden, es sencillo rastrear el flujo de la aplicación y aislar fallos. Este patrón es análogo al concepto de *Composition Root* empleado en arquitecturas más formales como la Inyección de Dependencias.

La integración de APIs externas (Open-Meteo y Geolocation) con **manejo explícito de errores y fallbacks** demostró la importancia de diseñar para la falla: una aplicación web real debe funcionar de forma degradada cuando los servicios externos no están disponibles o el usuario deniega permisos, y este proyecto lo contempla en todos los módulos que interactúan con recursos externos.

Finalmente, la refactorización documentada en la rama `feature/testing` evidenció el valor del proceso iterativo: módulos que nacieron acoplados y con lógica mezclada fueron progresivamente depurados en funciones puras, desacopladas y con responsabilidades únicas, resultando en un código final más legible, mantenible y extensible.